CSCE 4930 - Hardware Architecture for AI and ML

Fall 2026

Project: Hardware-Aware ML Deployment on the SparkFun Edge

1. Background

A model developed on desktop-class CPUs and GPUs can behave differently on a resource-constrained processor because of limits in compute, memory capacity, memory traffic, and supported numerical operations.

This project uses an accelerometer-based gesture-recognition model on the SparkFun Edge (Ambiq Apollo3 Blue, Arm Cortex-M4F). The work covers FP32 and INT8 deployment, memory and performance characterization, bottleneck analysis, and hardware-aware optimizations.

2. Model and Dataset

Dataset: Magic Wand accelerometer dataset (link)

The dataset consists of four directories: one for each gesture (wing, ring, and slope) plus a negative directory for data that represents no distinct gesture. Each directory contains files representing raw data from the gesture capture process.

For each gesture, the files correspond to named individuals, with the last part of the filename corresponding to the user ID. There are approximately 15 individual performances of a given gesture in each file, with one accelerometer reading per row. Each performance is prefixed by -,-,-.

Each performance contains up to a few seconds of data at 25 rows per second. The gesture occurs at some point within that window, with the device held still for the remainder of the time.

Participant-based data split:

- **Training:** 434 examples (63.7%); hyw, shiyun, tangsy, dengyl, jiangyh, xunkai; negative 3.
- **Validation:** 92 examples (13.5%); pengxl; negative 2 and 4.
- **Test:** 155 examples (22.8%); liucx, zhangxy; negative 1 and 5.

Baseline architecture:

- **Input:** 128 x 3 x 1
- **Conv2D:** 8 filters, 4 x 3, stride 1, same padding, ReLU
- **MaxPool2D:** 3 x 3, stride 3 x 3, valid padding
- **Dropout:** 0.1
- **Conv2D:** 16 filters, 4 x 1, stride 1, same padding, ReLU
- **MaxPool2D:** 3 x 1, stride 3 x 1, same padding
- **Dropout:** 0.1
- **Flatten**
- **Dense:** 16, ReLU
- **Dropout:** 0.1
- **Dense:** 4, Softmax

The FP32 baseline must achieve at least **90%** accuracy on the held-out test set.

3. Reference Environment

Use the following reference environment for the project:

- **TensorFlow / TensorFlow Lite Micro:** Commit 919f693420e35d00c8d0a42100837ae3718f7927
- **GNU Arm Embedded Toolchain:** Release 10-2020-q4-major

- **AmbiqSuite:** Version Rel2.2.0
- **Linker script:** tensorflow/lite/micro/tools/make/downloads/AmbiqSuite-Rel2.2.0/boards/apollo3_evb/examples/hello_world/gcc_patched/apollo3evb.ld
- **FlatBuffers:** Version 1.12.0

4. Project Requirements

1. FP32 Baseline Deployment and Functional Validation

- Train and evaluate the FP32 baseline model and deploy it on the SparkFun Edge using TensorFlow Lite Micro.
- Characterize the deployed TFLite graph and relate the operators that execute on the MCU to the original model, including operators that are fused, removed, or represented differently.
- Compare desktop and on-device outputs, predictions, and accuracy using the same held-out dataset inputs, and explain any differences.

2. FP32 Memory Characterization

Use a fixed TFLM tensor arena of 60 KiB (61,440 bytes).

- Characterize the final linked Flash/SRAM memory layout and placement, showing the available capacities and the sizes and placement of the major components (e.g., the deployed model and tensor arena), and comment on how the memory regions are used (e.g., occupied or available space).
- Relate the generated binary image to the linked Flash layout, showing the breakdown of the application Flash footprint, its utilization of the available Flash, and commenting on the result.
- Characterize the TFLM tensor-arena usage and memory plan, showing how the arena requirement is formed and how it relates to the theoretical model memory requirement (e.g., through tensor lifetime and reuse).
- For every deployed operator, compare the theoretical live FP32 activation memory with the TFLM-planned non-persistent memory extent at that execution step. Show the calculation and explain any differences.
- Identify where the peak transient SRAM requirement occurs during inference and explain why.
- Characterize the peak application SRAM requirement during inference and relate it to the linked SRAM footprint.

3. Model Storage, Computational Work, and Execution Cost

- Calculate parameter count, parameter storage, and MAC count where applicable for every deployed operator and the full model. Show representative calculations.
- Compare the operators with the largest parameter storage and largest MAC count, and explain whether and why they differ.
- Compare the calculated parameter storage with the serialized TFLite model size, characterize the remaining model storage, and comment on its sources.
- Quantify the model's contribution to the firmware Flash footprint and characterize the main non-model contributions.
- Measure and characterize full-model and per-operator inference latency and processor cycles.
- Compare each operator's theoretical computational work with its measured execution cost, using cycles per MAC where meaningful, and explain any differences, including operators with few or no MACs.

4. Hardware Characterization and Bottleneck Analysis

- Characterize the relevant on-device compute and memory performance limits and document the measurement conditions.
- For every deployed operator, develop an analytical performance model from its theoretical work and relevant data movement, predict the dominant limitation, and compare it with measured behavior.
- For each distinct deployed operator type, use its implementation to explain the measured behavior and any differences from the analytical model.

5. INT8 Deployment and Comparison

Deploy two INT8 versions of the baseline:

- Quantization-aware training (QAT-INT8)
- Post-training quantization (PTQ-INT8)
- Repeat the applicable characterization and analysis from Requirements 1-4 for both quantized models.
- Compare FP32, QAT-INT8, and PTQ-INT8, explain the trade-offs and any differences, and justify which model provides the best overall deployment choice.

6. Hardware-Aware Optimizations

- Investigate at least two materially different hardware-aware optimizations motivated by the preceding findings and any further analysis you consider necessary.
- For each optimization, justify its selection and predict the expected effect before implementation.
- Implement and evaluate each optimization against the selected model, compare the measured outcome with the prediction, and explain any differences using relevant Apollo3/Cortex-M4F architectural characteristics. Performance improvement is not required; correct analysis is.

5. Deliverables

- A GitHub repository containing the source code, measurement/analysis scripts, supporting result files, and all other project deliverables.
- A README describing the repository contents, with a brief description of each file, and how to build, run, and reproduce the submitted results.
- A report in PDF format presenting the required measurements, calculations, results, analysis, and conclusions. Implementation and reproduction details should remain in the repository/README unless they are needed to explain a result.
- The report must use 1.15 line spacing and font size 11 for the text.
- The report cover page must show the course name, course number, semester, and group members' names.
- All graphs must be well labelled.

6. Grading

- FP32 deployment and analysis [40%]
- INT8 deployment and comparison [25%]
- Hardware-aware optimizations [25%]
- Report and repository quality [10%]

Credit is based on demonstrated understanding, not final values alone. Show the calculations, measurements, breakdowns, comparisons, and reasoning needed to support the reported results.

Relevant supporting detail is encouraged; missing information needed to justify a result may be penalized.

Each group member is responsible for understanding the complete project and the concepts, implementation decisions, measurements, and analysis related to the requirements, including relevant questions that may arise during project discussions even if they are not explicitly listed in the specification.

Requirement-specific deficiencies are graded under the corresponding technical category, regardless of whether they appear in the implementation, measurements, analysis, report, README, repository, or interview. The Report and repository quality category covers overall documentation, organization, presentation, and submission quality that is not specific to one technical requirement.

- GitHub Development Compliance: -10% if GitHub is not used for project development.

7. Guidelines

- Team Formation: Work in groups of three students.
- Version Control and Workflow: Use GitHub for development. Accept the invitation to the assignment here. Maintain a disciplined workflow, as the commit history and individual contributions will be evaluated. Avoid informal methods (e.g., WhatsApp file sharing) for project development.
- Keep the repository well-structured and include the source, scripts, data/results, and other files needed to reproduce the submitted analysis.
- The source code must be well-commented, and files must be appropriately named.
- Document the measurement configuration needed to reproduce the reported results.

8. Milestones

- Team formation, group registration, and creating the repository by accepting the assignment: October 7 (-10% if not done on time)
- Intermediate interview; FP32 part (Requirements 1-4) completed and submitted with the fp32 tag: November 2
- Final submission using the final tag: December 6
- Final interviews during class time: December 7 and December 10

References

- SparkFun Edge Hookup Guide: https://learn.sparkfun.com/tutorials/sparkfun-edge-hookup-guide
- SparkFun Edge Schematic: https://cdn.sparkfun.com/assets/2/b/7/4/d/SparkFun_Edge_Schematic.pdf
- Apollo3 Blue SoC Datasheet: https://ambiq.com/wp-content/uploads/2020/10/Apollo3-Blue-SoC-Datasheet.pdf
- SparkFun AmbiqSuite SDK: https://github.com/sparkfun/AmbiqSuiteSDK
- Arm Cortex-M4 Technical Reference Manual: https://developer.arm.com/documentation/100166/latest/
- TensorFlow Lite Micro: https://github.com/tensorflow/tflite-micro
- TensorFlow Lite Micro Documentation: https://github.com/tensorflow/tflite-micro/blob/main/tensorflow/lite/micro/docs/
- TensorFlow Lite Experimental API: https://www.tensorflow.org/api_docs/python/tf/lite/experimental
- TensorFlow Lite operator definitions: https://www.tensorflow.org/mlir/tfl_ops

- TensorFlow Lite schema: https://github.com/tensorflow/tensorflow/blob/master/tensorflow/compiler/mlir/lite/schema/schema.fbs
- GNU linker documentation: https://sourceware.org/binutils/docs/ld/
- GNU Binary Utilities documentation: https://sourceware.org/binutils/docs/binutils/
- LiteRT Model Optimization: https://developers.google.com/edge/litert/quantization/model_optimization
- TensorFlow Model Optimization Guide: https://www.tensorflow.org/model_optimization/guide
- NERSC Roofline Performance Model: https://docs.nersc.gov/tools/performance/roofline/